

ForkNFind Functional Specification

By Eoin Daly & Thomas Hazekamp

Table of Contents

	1
1. Introduction	2
1.1 Overview	2
1.1.1 Sentiment Analysis	2
1.1.2 Hybrid Recommendation System	3
1.1.2.1 Collaborative filtering	3
1.1.2.1 Content based filtering	3
1.2 Business Context	4
1.3 Glossary	4
2. General Description	5
2.1 System Functions	5
2.2 User Characteristics and Objectives	5
2.3 Operational Scenarios	6
2.3.1 Unregistered User	6
2.3.2 Registered User	6
2.4 Constraints	6
2.4.1 Time	6
2.4.1 Hardware	7
3. Functional Requirements	7
3.1 Create Account	7
3.2 Login	7
3.3 Logout	8
3.4 Change Settings	8
3.5 Map View	9
3.6 Recommended Restaurants	9
3.7 Search for Restaurants	10
3.8 Post Review	11
3.9 Sentiment Analysis	11
3.10 Hybrid Recommender System	11
3.11 Delete Account	12
4. System Architecture	13
4.1 Client Layer	13
4.2 Frontend	13
4.3 Backend	13
4.4 Data Storage Layer	13
4.5 Database Schema	14

5. High Level Design	15
5.1 Data Flow Diagram	16
5.2 Use Cases	18
5.2.1 Logging In	18
5.2.2 View Map recommendation	19
5.2.3 Create Review	20
5.3 Context Diagram	21
6. Preliminary Schedule	22
7. Appendices	22

Student 1:	Eoin Daly - 20424366
Student 2:	Thomas Hazekamp - 20423602
Supervisor:	Brian Davis
Date:	17/11/2023

1. Introduction

1.1 Overview

ForkNFind is a mobile application for anyone looking to find their next place to eat. Using sentiment analysis and a recommendation system our mobile application will look to pair users up with restaurants that suit them.

We identified a problem that while working on our **INTRA** internships we found it challenging to be able to pick places to eat that would suit our individual tastes and preferences. Due to our busy workloads we would have benefitted from having access to an efficient application like the one we plan on developing.

ForkNFind is our solution to the above mentioned problem and this will allow for a quicker informed decision from its users as to where to go for their next restaurant of choice. At the centre of our solution to this problem is to make use of two machine learning techniques: a **hybrid recommendation system** and sentiment analysis. A brief description of our planned models,

1.1.1 Sentiment Analysis

Sentiment analysis will allow us to understand the attitude towards specific reviews, which will then help us with a more accurate recommendation of particular locations. Sentiment analysis is fundamental in the **natural language processing** world, this will take a users review, positive or negative and provide a classification for it. As the book [5] states, real-world sentiment analysis is moving away from positive or negative and towards the

emotion behind the text. We plan on implementing more of an emotion-based sentiment analysis, this can prevent the issue of a negated negative response being considered positive [5]. Some libraries or packages we will be considering include but are not limited to: **Stanza** [6], **VADER** [4] and **BERT** [2][3] (using transformers library).

1.1.2 Hybrid Recommendation System

Our recommendation system will be built by combining both a i) **content based filtering model** and ii) **collaborative based filtering model**.

1.1.2.1 Collaborative filtering

Collaborative filtering is a method of filtering designed to compare how similar user's are based on how they have been interacting and engaging with items. It works by getting a large group of users and looking at how they have been rating items, historical preferences and interactions. With this user information it finds similar users based on how they have been rated items, historical preferences and interactions. Once users similar to a specific user have been found then it can recommend some of those items that they have rated highly. A simplified example is if User A likes a restaurant and User B also likes that restaurant, then if User B likes another restaurant that User A has not been to we can recommend that restaurant to User A.

There are many libraries in Python that allow for the creation of collaborative filtering models, like the python library Surprise. The dataset would be fitted to a Surprise model that uses one of many possible machine learning algorithms like KNNbasic, KNNWithMeans, Singular Value Decomposition (SVD), Slope One. These algorithms would all use the historical user-item interactions to make predictions on unseen items, each one attempting a new unique approach for capturing the relationships.

The main library we want to focus on is the Python LightFM library, this library is designed for building hybrid recommenders but as collaborative filtering is a part of a hybrid recommender it also has the tools to build this type of recommender. LightFM for its collaborative filtering uses an interaction matrix, where each entry represents a user's rating for an item. Using machine learning algorithms like WARP, logistic and BPR to capture the relationships between positive and negative interactions between user and items.

1.1.2.1 Content based filtering

Content based filtering is designed to recommend new items to users based on existing items they have had positive interaction with. Content based filtering uses features from the items such as keywords and attributes to compare and find which items are similar to each other. An example of a book having the genres 'thriller' and 'murder mystery' would be judged similar to another book with the genre 'thriller' compared to a book on bird watching.

A library that can be used for building a content based recommendation system is the python library scikit-learn. Scikit-learn can be used using its inbuilt function TfidfVectorizer to develop a content based recommendation system. Using this function we could transform textual information, like a product description, into vectors emphasising terms that are frequent but not common across all documents. The resulting vectors can be compared

using the cosine similarity to get a value for how similar they are, and once comparing across all vectors we would get to know which items are most similar based on the items content.

LightFM can also be used to generate a content based recommendation system, while the library isn't specifically designed to build a content based recommender itself, rather designed to build it into a **hybrid recommendation system** it can be accomplished. This can be accomplished by adding associated tags and labels that would be associated with each item to a matrix where each item would have its value for that tag or feature. During the training of a model, the item features would be incorporated into the prediction process where they would then be used in testing.

1.2 Business Context

The possible business context for this mobile application, including future revenue plans, target audience and how we would assess the performance of our goals:

Revenue:

- Advertising on the mobile app could generate revenue for the company.
- Having a membership scheme for restaurants allows them to appear more prominently throughout searches or on an app.
- Selling this app to be integrated into another pre-existing application.

Target Audience:

- People looking to explore new establishments & experiences
- The age demographic would be teenagers and adults
- People looking to provide other people with reviews about their experiences

Goal performance:

- Obtaining user feedback
- Obtaining numbers on possible financial success
- How efficient is the application from a users perspective

1.3 Glossary

- **INTRA - INTeGrated TRAIning**, this is DCUs work experience name.
- **Natural Language Processing** - a field in Artificial Intelligence, which allows computers to understand and use human language.
- **Django** - Python based web framework, it allows for users to rapidly develop and maintain their applications
- **API** - a way in which two or more computers can communicate with each other using set routines and protocols
- **Content-based system** - a type of system which uses items and users
- **Collaborative system** - a type of system in which the system is based on the behaviour of the user
- **Hybrid recommendation system** - a recommendation system that uses both collaborative and content based data
- **React Native** - open-source UI software made for mobile applications
- **Backend** - relates to the database and logic of a system

- **Frontend** - the interaction between the system and the end user
- **Stanza** - A Python natural language analysis package
- **VADER** - A rule-based sentiment analysis tool
- **BERT** - A sentiment analysis tool

2. General Description

2.1 System Functions

Below are the main functions that we plan to implement into the application. Over the next few months these functions might change as this initial list is open to change.

Function	Description
Create an Account	User can create an account inside the mobile application
Login to Account	User can login the created account inside the mobile application
Logout of Account	User can logout of the account
Map view of nearby restaurants	User can view a map with food spots
Recommendations of restaurants	User will receive recommendations for restaurants based on previously given reviews and other users reviews
Search for restaurants	User will be able to search for specific restaurants
Personal Settings	User will be able to view and change personal settings
Leave a review for a restaurant	User will be able to leave a review for any restaurant they have visited

2.2 User Characteristics and Objectives

Since dining out in a restaurant is available to everybody, this is something we have to keep in mind when designing our application. Whether the user is familiar with using technology or technologically inexperienced we want them to be able to pick up our application and be able to use it with no challenge. We see the target audience of the user's being people that just want to go out and have some food at a place they would enjoy.

The primary objective of the system from a users perspective is to be able to easily open the application and find a restaurant that they would enjoy. At the same time, a secondary objective is to make the mobile application easy to use.

2.3 Operational Scenarios

2.3.1 Unregistered User

- Map View
 - The user will be able to access the map view of the surrounding area to view what restaurants are nearby.
- Search
 - The user will have the search accessible to look up based on filters like, type of food served, price range, distance or to search specific restaurants.
- Create an Account
 - The user will have the ability to create an account on the application to have access to user specific ability, i.e. recommendations.

2.3.2 Registered User

- Login
 - The user will be able to login to their ForkNFind account to access user specific functionality.
- Map View
 - The user will be able to access the map view of the surrounding area to view what restaurants are nearby.
- Recommendations
 - The user will be able to get recommendations for restaurants that it is believed they would rate positively.
- Search
 - The user will have the search accessible to look up based on filters or to search specific restaurants.
- Settings
 - The user will be able to change settings about their account like resetting their recommendation profile.
- Logout
 - The user will be able to logout of their ForkNFind account.

2.4 Constraints

2.4.1 Time

While we both have previous experience with the third year project and time management, this is a different challenge due to the fact we also have modules running concurrently while we are meant to be building and designing the project. Managing the time of the project whilst also juggling our other modules will be a challenging part of the project.

2.4.1 Hardware

Whilst building and testing our machine learning parts of the project we will be somewhat limited to the hardware that we have available at our disposal. We will have to manage the size of the datasets as increasing the datasets will heavily impact the performance and the amount of time it takes to train and test models.

3. Functional Requirements

3.1 Create Account

Description

First time users that will have the opportunity to create an account giving them access to the mobile application. The registration will require users to fill in their information first name, second name, username, email address and password.

Criticality

This functionality will be of utmost importance as without this functionality users will be unable to have access to the recommender aspects of the mobile application.

Technical Issues

The account verification will be handled by **Django** User Authentication which is **Django's** built-in authentication system, each username will have to be unique.

Dependencies

This functionality is not dependent on any other requirement.

3.2 Login

Description

Once a user has successfully registered an account, they have the opportunity to login to the mobile application using their username and password. The details that have been inputted will both be checked to make sure that both are a valid username and password and if successful they will be logged into the mobile application. If the details are incorrect then the user will be unavailable to access the account.

Criticality

This functionality is essential for security and for accessing the recommender aspects of the mobile application. This will unauthorised access to the mobile application.

Technical Issues

The account verification will be handled by **Django** User Authentication which is **Django's** built-in authentication system.

Dependencies

This functionality is dependent on the create account functionality as without a user having an account this wouldn't be possible.

3.3 Logout

Description

Once a user has successfully logged into the mobile application, they need to have the ability to click a button to logout of the mobile application.

Criticality

This functionality is essential for security as not having the ability to logout of the mobile application would be an issue.

Technical Issues

The account verification will be handled by **Django** User Authentication which is **Django's** built-in authentication system.

Dependencies

This functionality is dependent on the create account functionality as without a user having an account this wouldn't be possible and is also dependent on the fact a user is logged in.

3.4 Change Settings

Description

Once a user has successfully logged into the mobile application, if they want to change any account related information or delete their current recommendation history they can do this from the settings menu.

Criticality

This functionality is not essential as users would have already inputted their information in the create account screen.

Technical Issues

No technical issues.

Dependencies

This functionality is dependent on the user being logged into the mobile application.

3.5 Map View

Description

The home page of our application will be designed to show a map of the user's current location and any restaurants in the nearby vicinity to be highlighted. A user-friendly interface will be important for this part of the application, as if it was clustered and messy it would be an eyesore.

Criticality

This functionality is important as this will allow the user to see restaurants in the nearby area and allow them to click on the restaurants.

Technical Issues

To get the restaurants in the nearby area we will have to use the Google Maps **API** to get the information about the restaurants in the nearby vicinity, specifically getting the precise location of the restaurants will be important.

Dependencies

This functionality is not dependent on any other requirement.

3.6 Recommended Restaurants

Description

Similarly to the above mentioned Map View this will be the same concept of having a map of the users location with the nearby restaurants, but crucially this will be for logged in users allowing them to see specifically only restaurants which have been recommended to them as a place they would rate positively.

Criticality

This functionality is important as this will allow the user to see restaurants in the nearby area and will showcase the recommendation systems tailored suggestions based on user preferences, providing personalised restaurant recommendations, such as hiding restaurants from the view in the current area due to the fact that they would not be recommended to the user.

Technical Issues

Using the Google Maps **API** to only show the nearby restaurants while also checking if each of those restaurants would be recommended by the recommendation system.

Dependencies

This functionality is dependent on the user being logged into the mobile application.

Design

A sample UI design shows how we would want to show the locations of the restaurants with an icon. These restaurants would be recommended for the user to visit.

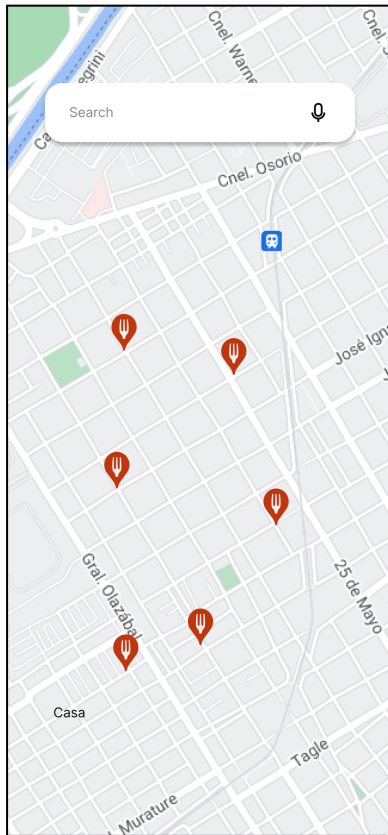


Fig 1. UI design (Figma - [7])

3.7 Search for Restaurants

Description

Users will have the ability to use a search bar to search for restaurants. This will also include filters so that they can filter by specific restaurant type or other features.

Criticality

This functionality will be important as it will allow users to find restaurants outside of the map view screen which will only be a small area around them.

Technical Issues

Creating a search engine will require us to query the database to return the results.

Dependencies

This functionality is not dependent on any other requirement.

3.8 Post Review

Description

Users will have the ability to leave a review for a restaurant, which can be done by clicking onto the restaurant's home page and clicking a leave review button.

Criticality

This functionality will be important as it will be used as part of the recommendation system to see how users are feeling after being in a restaurant.

Technical Issues

No technical issues.

Dependencies

This functionality is dependent on being able to see restaurants in both the map view and the search functionality.

3.9 Sentiment Analysis

Description

Document level analysis will be used to determine whether a users review is negative or positive, this will allow for us to determine a rating for the reviewed restaurant.

Criticality

This functionality will be important as it will be used by the recommendation system to determine accurate recommendations.

Technical Issues

Sentiment analysis will leverage existing **APIs** for train state of the art tools based on machine learning techniques. This will involve testing using metric based approaches and human evaluation. We will have to research the best methods and packages to use with our desired analysis.

Dependencies

This feature will depend on the mobile application review functionality, as for this feature to work correctly, it will need users to leave reviews on restaurants.

3.10 Hybrid Recommender System

Description

The hybrid recommender system will be where all the machine learning for generating which restaurants we are going to recommend to a user. This will be done using an item-to-item

similarity using the restaurants and a user-to-user similarity using the users of the mobile application.

Criticality

This functionality will be critical to the system as without it we won't be able to give user recommendations.

Technical Issues

As the Hybrid Recommendations are a new machine learning technology there will be a lot of learning and testing involved specifically in this area, which will be both a technical issue and challenge.

Dependencies

This relies on being able to leave reviews for restaurants and also dependent on the sentiment analysis output working so it can be run on the reviews to see if they are positive or negative.

3.11 Delete Account

Description

A user will have the ability to delete their account if they want to stop using the application.

Criticality

This functionality is not important for the overall system.

Technical Issues

Correctly deleting all the user data relating to the user when they go to delete their account.

Dependencies

The user must have both an account created and be logged into their account when attempting to delete the account.

4. System Architecture

This system architecture diagram shows a high-level structure of our designed system, it outlines how our system is designed to interact with all the subsystems we will be building.

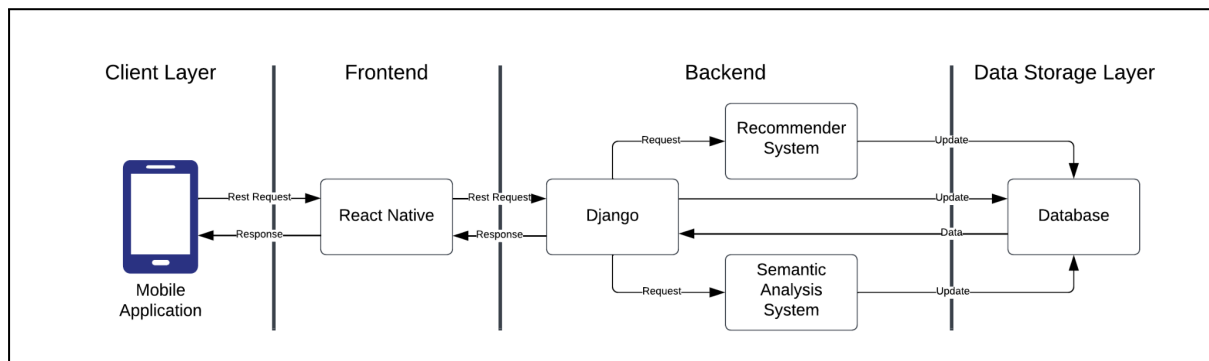


Fig 2. System Architecture Diagram

4.1 Client Layer

The mobile application will be the user facing component. The application will be both available on IOS and Android.

4.2 Frontend

The **React Native frontend** will enable us to develop a robust, responsive visual interface for the users to interact with. **React Native** allows us to develop a mobile application so that it can be deployed on both IOS and Android. The component nature of **React Native** and its various third party libraries makes it a great tool to develop with. It will be communicating with the **Django backend** using various **API** calls.

4.3 Backend

The **Django backend** gives us the opportunity to rapidly develop a clean and pragmatic **backend** for our system. We will be able to design the **API** endpoints using **Django** which will allow for the seamless communication of data from **frontend** to **backend**. With built in user authentication that can be easily integrated for security. **Django** will also be communicating with both the recommender system and the semantic analysis system, this can be achieved as when new information is being received by **Django** it can call both these systems to update the current user profiles for the recommender and analysis user reviews for the semantic analysis. All the information can then be updated in the database.

4.4 Data Storage Layer

The database will be used to store all user information, restaurant information and the machine learning models. **Django** can update and query the database to get information.

4.5 Database Schema

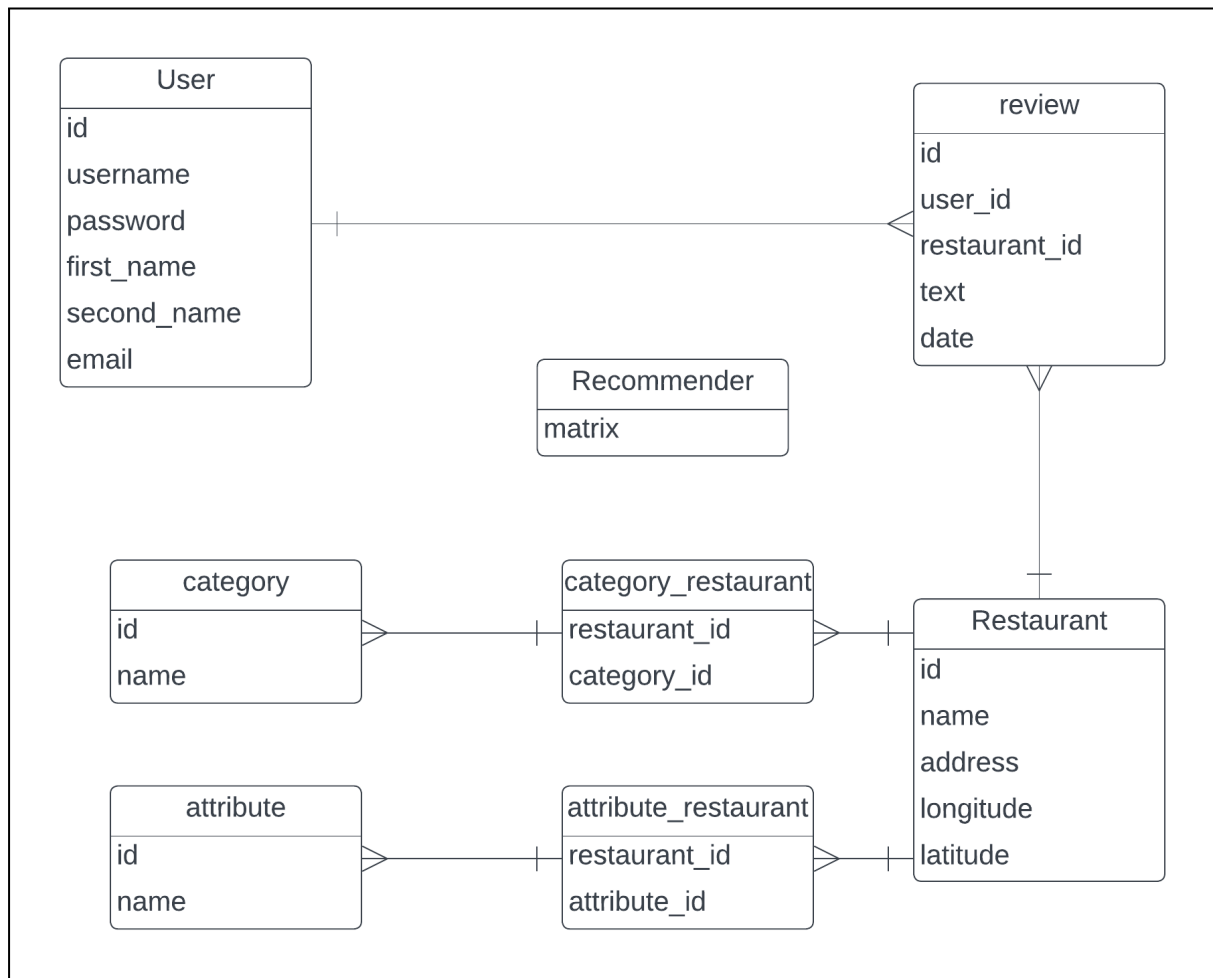
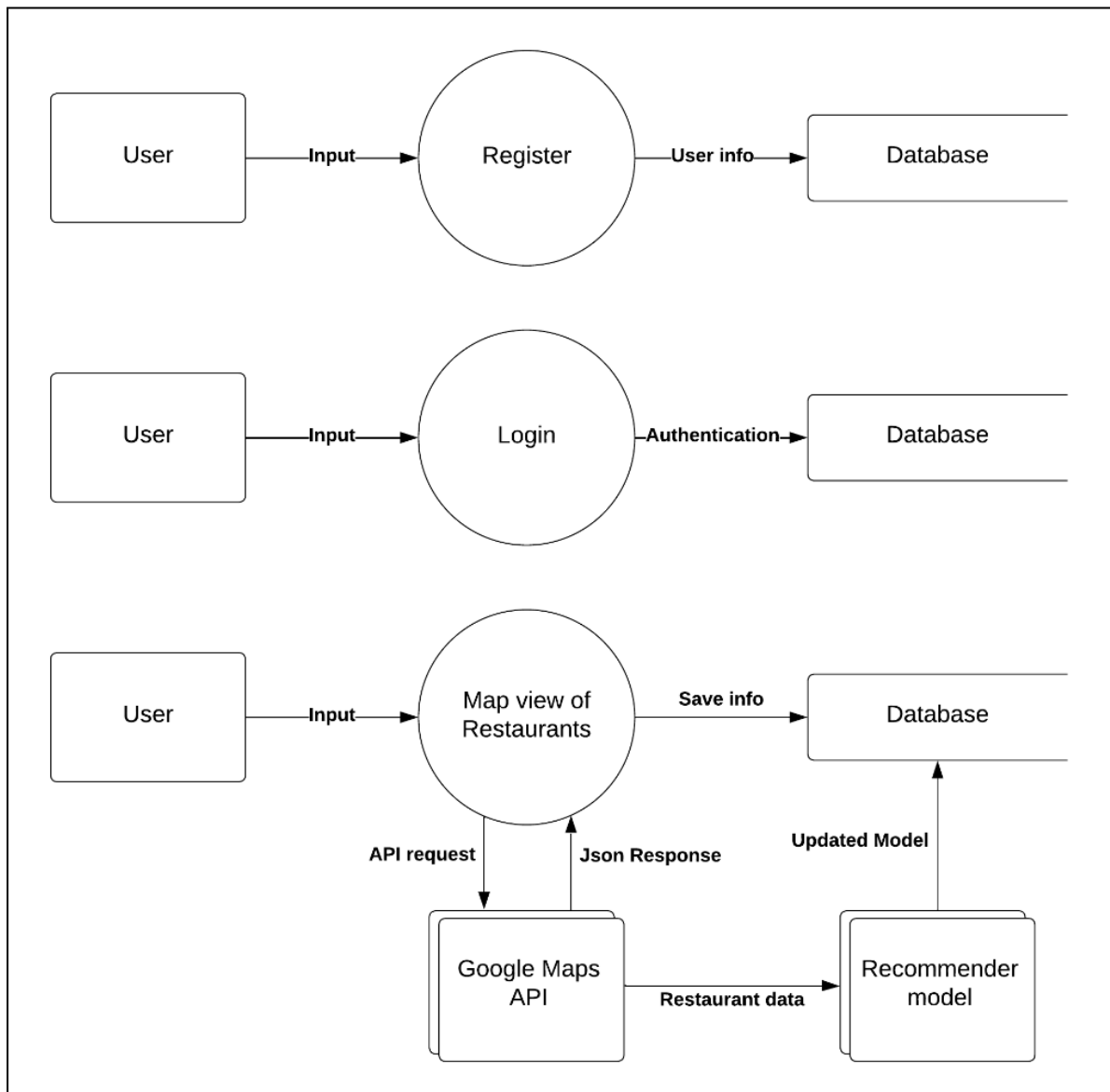


Fig 3. Database Schema

5. High Level Design

5.1 Data Flow Diagram



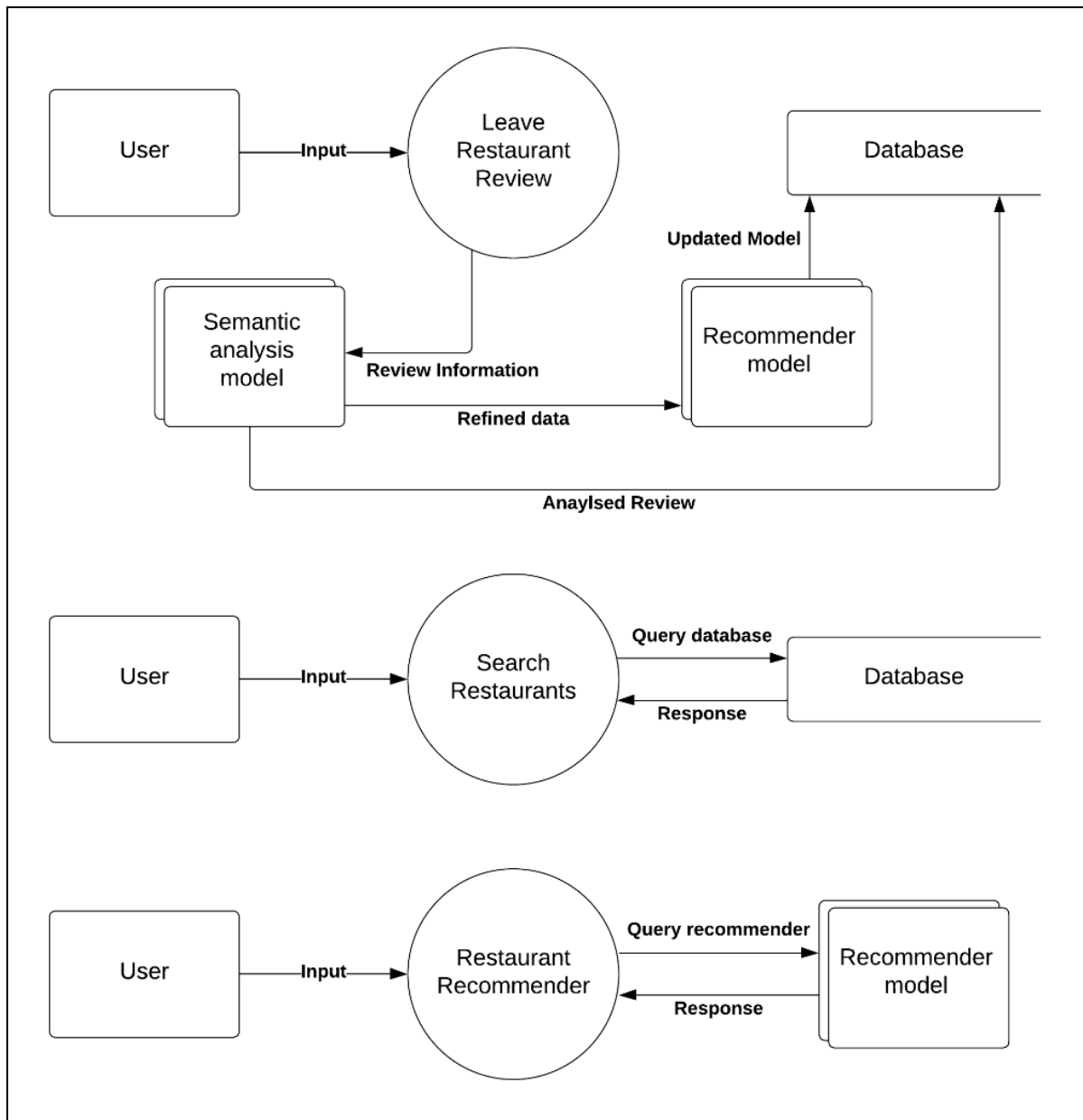


Fig 4. Data Flow Diagram

5.2 Use Cases

5.2.1 Logging In

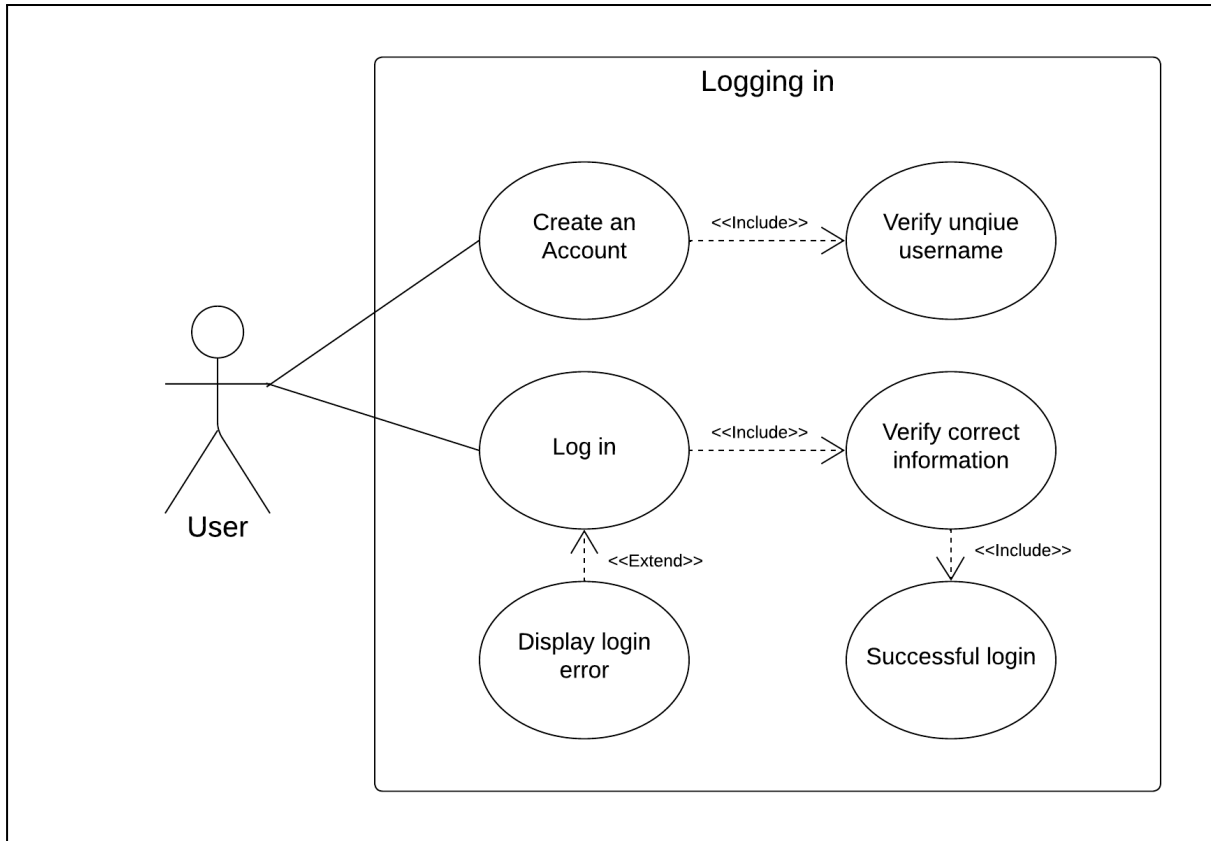


Fig 5. Login Use Case Diagram

5.2.2 View Map recommendation

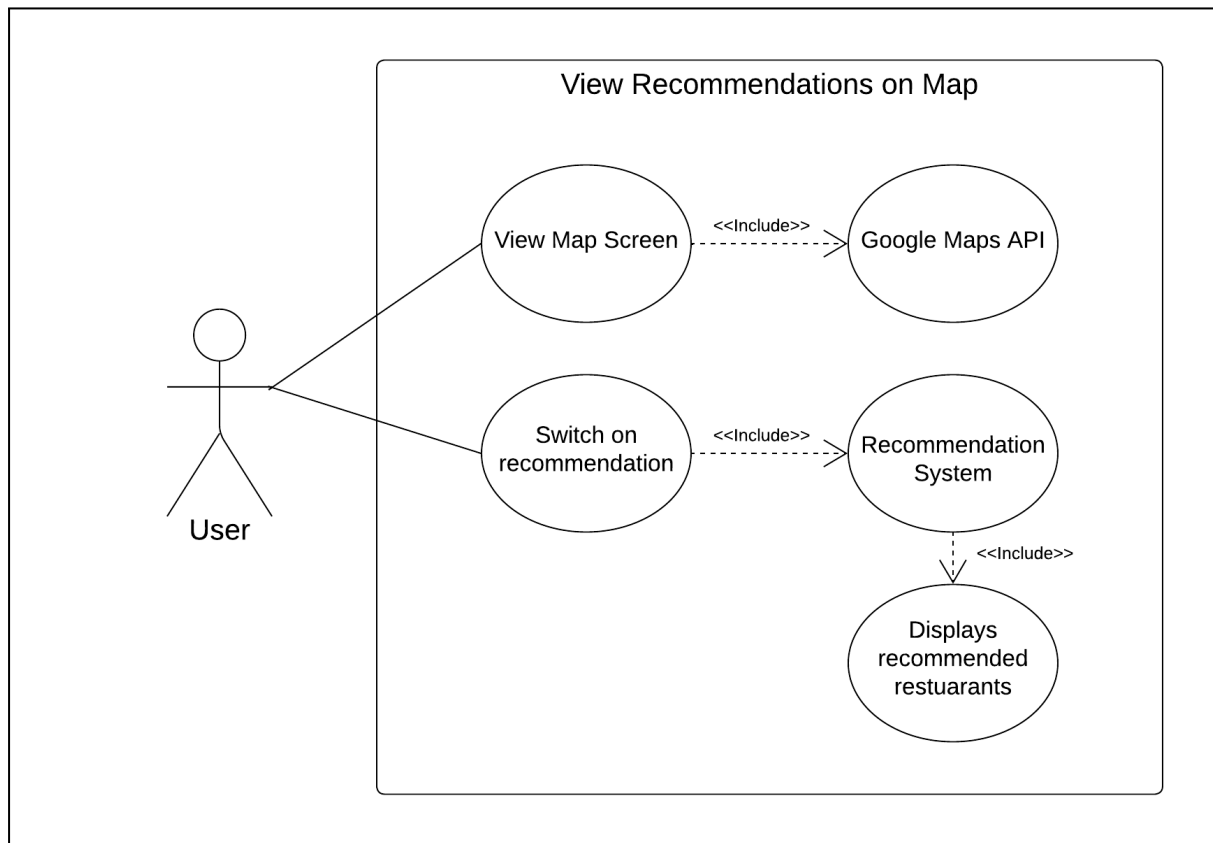


Fig 6. View Map Recommendation Use Case Diagram

5.2.3 Create Review

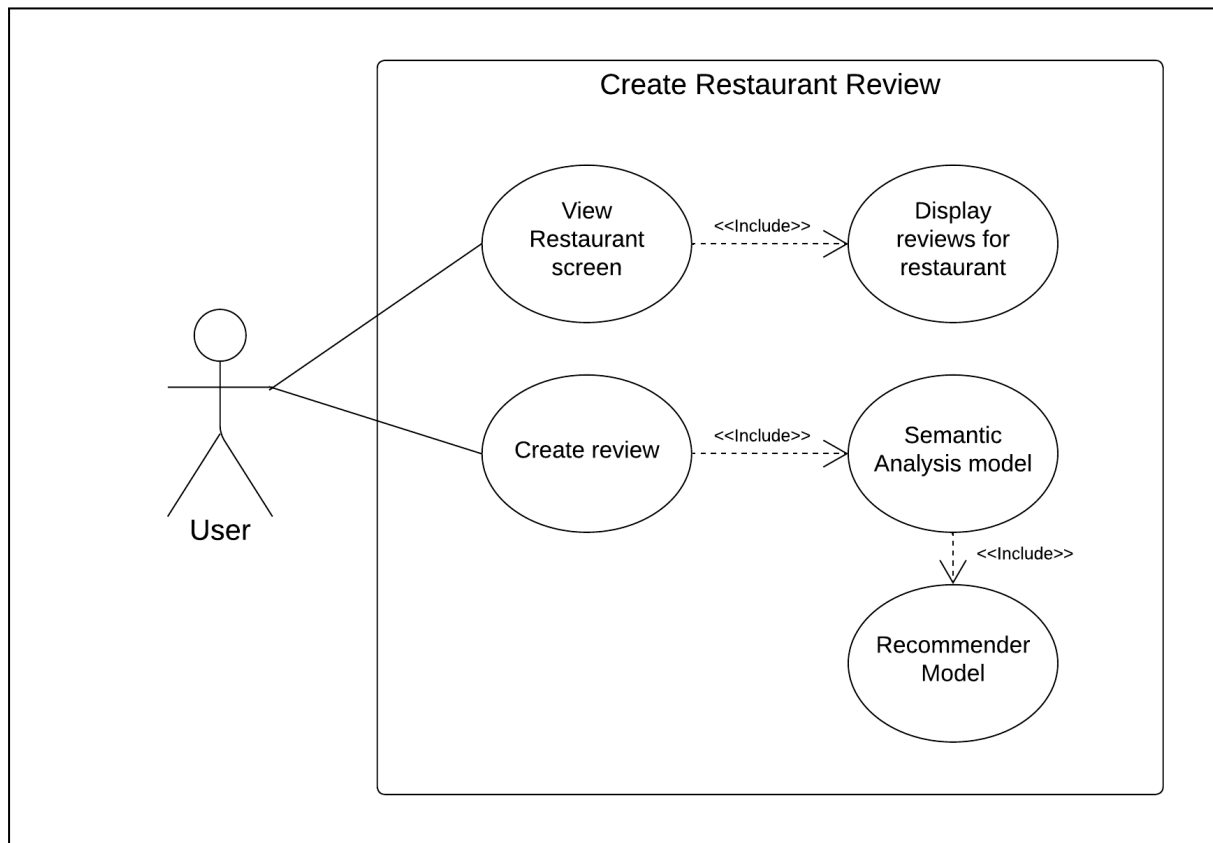


Fig 7. Create Review Use Case Diagram

5.3 Context Diagram

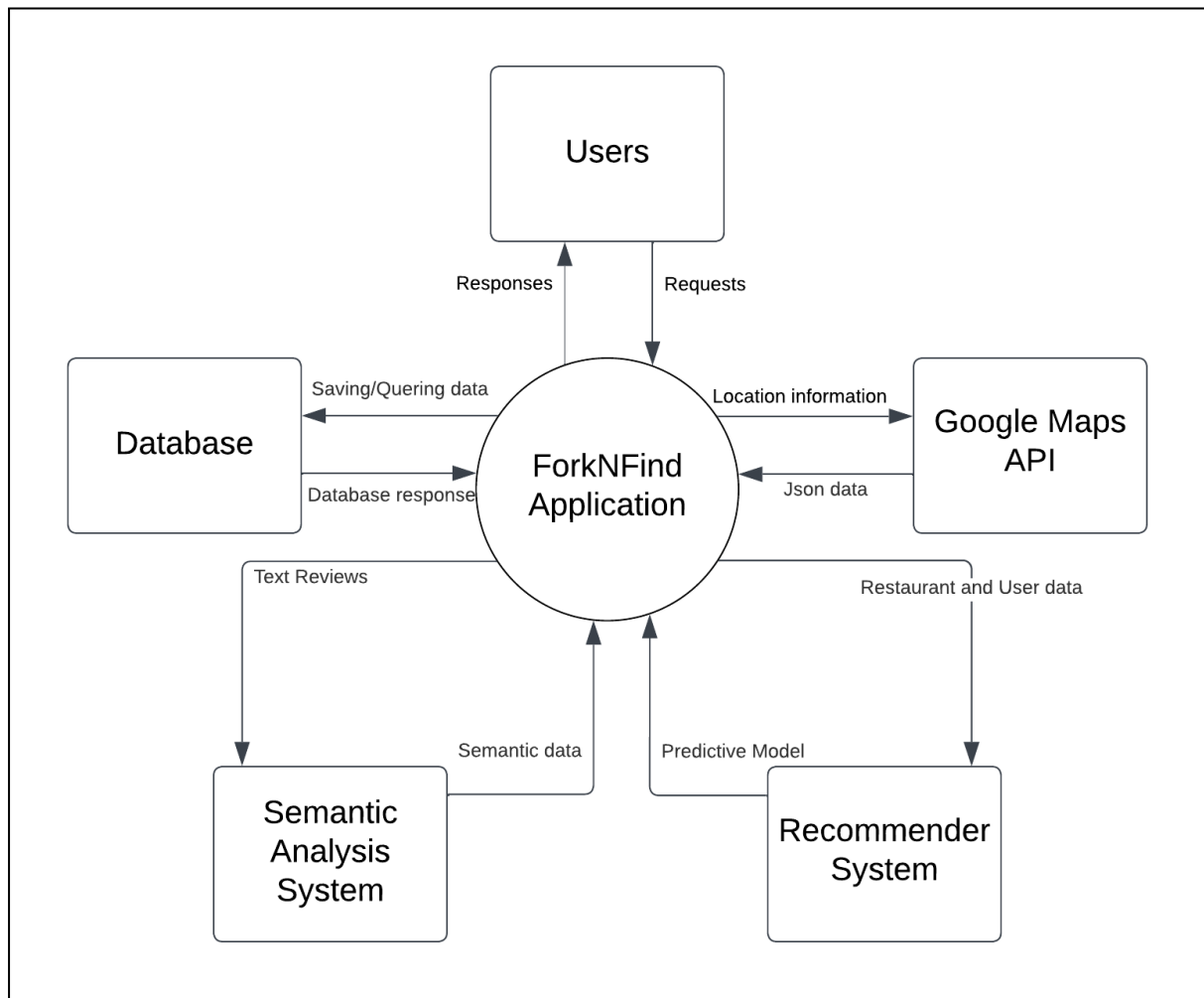


Fig 8. Context Diagram

6. Preliminary Schedule

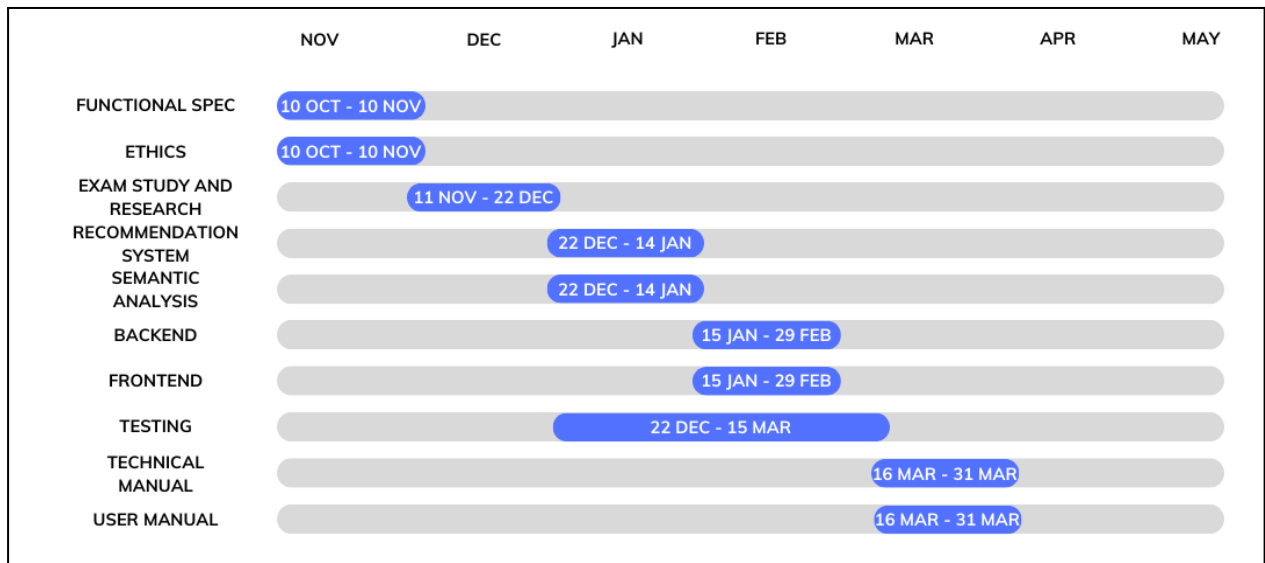


Fig 9. Preliminary Schedule Diagram

7. Appendices

- [1] *React*, <https://react.dev/>. Accessed 16 November 2023.
- [2] “V.” *YouTube*, 16 June 2023, <https://huggingface.co/transformers/v2.2.0/index.html>. Accessed 16 November 2023.
- [3] “Google Colab.” *Google Colab*, <https://colab.research.google.com/drive/1f32gj5lYlyFipolNiC8P3DvKat-WWLUK#scrollTo=IT93d88Pcep>. Accessed 16 November 2023.
- [4] Hutto, CJ, et al. “cjhutto/vaderSentiment: VADER Sentiment Analysis. VADER (Valence Aware Dictionary and sEntiment Reasoner) is a lexicon and rule-based sentiment analysis tool that is specifically attuned to sentiments expressed in social media, and works well on texts ...” *GitHub*, <https://github.com/cjhutto/vaderSentiment#introduction>. Accessed 16 November 2023.

[5] Maynard, Diana, et al. *Natural Language Processing for the Semantic Web*. Morgan & Claypool Publishers, 2016. Accessed 17 November 2023.

[6] "Overview - Stanza." *Stanford NLP Group*, <https://stanfordnlp.github.io/stanza/index.html>. Accessed 16 November 2023.

[7] "Figma." *Figma: The Collaborative Interface Design Tool*, <https://www.figma.com/>. Accessed 17 November 2023.